Round 2: Memory-Constrained Exact Attention

---

1. Problem Overview

The task is to compute the exact self-attention output for a sequence of vectors under strict memory constraints.

Given:

- Input: $X = [x_1, x_2, ..., x_n]$, each of dimension d
- Attention output for each vector:

y_i = sum over j of (softmax($x_i \cdot x_j^T$) × x_j)

The key constraint is that we **cannot store the full n × n attention matrix**, which makes the standard implementation infeasible.

---

2. Core Idea

Standard attention requires $O(n^2)$ memory due to storing all pairwise scores.

To overcome this, we use:

- **Chunked (tiled) computation**
- **Online softmax aggregation**
- **Recomputation of Attention Matrix**

Instead of computing all attention scores at once, we process them block-by-block and maintain running values for each query.

---

3. Forward Pass

Key Insight

Softmax can be computed incrementally using a running maximum and normalization factor to ensure numerical stability.

$$\boldsymbol{Algorithm}$$

$$1. Split\ input\ into\ blocks\ of\ size\ B$$

$$2. For\ each\ query\ block\ Q:$$

$$Initialize:$$

$$m = -\infty$$

$$l = 0$$

$$y = 0$$

$$3. For\ each\ key\ block\ K:$$

$$S = QK^T$$

$$m_{local} = \max(S)$$

$$m_{new} = \max(m, m_{local})$$

$$l = l \cdot e^{(m - m_{new})} + \sum e^{(S - m_{new})}$$

$$y = y \cdot e^{(m - m_{new})} + e^{(S - m_{new})} \cdot V$$

$$m = m_{new}$$

$$4. Final\ output:$$

$$\boldsymbol{y} = \frac{\boldsymbol{y}}{\boldsymbol{l}}$$

Memory Efficiency

- Only one block is stored at a time
- No full attention matrix is created
- Memory reduces from **$O(n^2)$ to $O(n \cdot d)$**

---

4. Backward Pass

Challenge

Standard backpropagation stores attention scores and softmax outputs, which is memory expensive.

Solution

We use **recomputation instead of storage**:

- Do not store attention scores
- During backward pass:
  - Recompute $Q \times K^T$ block-by-block
  - Recompute softmax locally
  - Use gradients to compute dQ, dK, and dV

Key Idea

- Store only inputs (Q, K, V) and final outputs
- Recompute intermediate values when needed

Trade-off

Aspect	Standard Attention	Proposed Method
Memory	$O(n^2)$	$O(n \cdot d)$
Compute	$O(n^2)$	Slightly higher due to recomputation

---

5. Optimizations

- Adaptive block size based on memory
- Fused operations (combine steps to reduce overhead)
- Mixed precision for faster computation

---

6. Conclusion

This approach computes exact attention without storing the full attention matrix by using tiled computation and online softmax.

The backward pass avoids storing intermediate activations through recomputation, significantly reducing memory usage while maintaining correctness.

This method is both memory-efficient and practical for large-scale sequence processing.